

동적 프로그래밍

벨만 방정식의 등호를 갱신식으로 바꾸기: 정책 반복법과 가치 반복법, 그리고 “평가를 얼마나 끝까지 돌리는가”라는 하나의 축

sajacaros
sajacall82@naver.com

2026년 8월 25일

초록

앞 장에서 벨만 방정식을 얻었지만 그것을 실제로 푸는 방법은 두 칸짜리 예제에서 손으로 연립방정식을 세우는 것뿐이었고, 직접 해법은 $O(|S|^3)$ 의 벽에 부딪힌다. 동적 프로그래밍(DP)은 이 방정식의 등호를 대입으로 바꿔 읽는다. 우변에 현재 추정치 V_k 를 넣어 좌변 V_{k+1} 을 얻기를 반복하면 참값과의 거리가 매번 γ 배 이하로 줄어들어 참값으로 수렴한다. 본고는 이 착상에서 출발하여 반복적 정책 평가, 정책 반복법, 가치 반복법의 세 알고리즘을 정리하고, 세 알고리즘이 “평가를 얼마나 끝까지 돌리는가”라는 하나의 축 위의 서로 다른 지점임을 보인다. 원서[1] 4장은 예제 코드를 제공하므로 본고는 그것을 그대로 쓰되, 원서가 관찰로만 남기거나 아예 짚지 않은 자리에 다섯 가지 논의를 더하였다. 절 구성은 원서 4장의 목차를 그대로 따르며, 더한 논의는 별도의 절로 떼지 않고 그것이 필요해지는 자리의 본문에 “[보충]”으로 표시해 두었다. 첫째, 원서가 풀지 않고 쓰는 “동적 프로그래밍”이라는 이름의 유래와 기법으로서의 정의를 밝히고, 반복적 정책 평가가 그 정의에 어떻게 대응하는지, 그리고 표를 한 번 채우면 끝나는 교과서 동적 계획법과 달리 되풀이가 필요한 까닭이 무엇인지를 적는다. 둘째, 배열 하나로 제자리 갱신하는 구현이 배열 둘을 쓰는 구현보다 빠른(76스왑 대 60스왑) 이유가 야코비 반복과 가우스-자이델 반복의 차이임을 밝히고, 정지 문턱을 낮출수록 그 이점이 커짐을 실측으로 보인다. 셋째, 정지 문턱 ϵ 으로 멈춘 값이 참값과 얼마나 떨어져 있는지를 따져 오차가 문턱 자체가 아니라 그 $\gamma/(1-\gamma)$ 배($\gamma = 0.9$ 에서 아홉 배)까지 허용됨을 보이고, 앞 장의 정확해 표와 이 장의 반복 계산 표가 세 칸에서 소수 둘째 자리까지 다른 것이 바로 이 오차임을 확인한다. 넷째, 정책 반복법과 가치 반복법의 비용을 같은 단위로 환산하여 평가 스왑 32회 대 4회, 백업 횟수로 1776회 대 240회임을 제시한다. 다섯째, 이 환경의 최적 가치가 $v_*(s) = \gamma^{d(s)-1}$ 이라는 닫힌 형태를 가짐을 보이고, 그로부터 가치 반복법이 문턱과 무관하게 유한 스왑 안에 끝나는 이유를 설명한다.

주제어: 강화 학습, 동적 프로그래밍, 반복적 정책 평가, 정책 반복법, 가치 반복법, 야코비 반복, 가우스-자이델 반복, 일반화된 정책 반복

차례

서론	2
4.1 동적 프로그래밍과 정책 평가	3
4.1.1 동적 프로그래밍 기초	3
4.1.2 반복적 정책 평가: 첫 번째 구현	6
4.1.3 반복적 정책 평가: 다른 구현 방법	7
4.2 더 큰 문제를 향해	10
4.2.1 GridWorld 클래스 구현	10
4.2.2 defaultdict 사용법	11
4.2.3 반복적 정책 평가 구현	12

4.3 정책 반복법	13
4.3.1 정책 개선	13
4.3.2 평가와 개선 반복	14
4.4 정책 반복법 구현	14
4.4.1 정책 개선	15
4.4.2 평가와 개선 반복	15
4.5 가치 반복법	17
4.5.1 가치 반복법 도출	17
4.5.2 가치 반복법 구현	18
4.6 정리	22
재현 정보	23

서론

연립방정식에서 반복 계산으로

앞 장[2]에서 벨만 방정식

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_{\pi}(s') \} \quad (1)$$

을 얻었고, 이것이 v_{π} 에 대한 연립일차방정식이므로 상태마다 식을 한 줄씩 세워 풀면 정확해가 나온다는 것도 확인하였다. 그러나 그 방법은 미지수를 정면으로 소거하는 일이라 연산량이 $O(|S|^3)$, 계수를 담은 표의 크기가 $O(|S|^2)$ 로 늘어난다. 상태 공간이 커질 때 먼저 무너지는 쪽은 시간이 아니라 공간이다.

그림 4-1 벨만 방정식을 이용하여 가치 함수를 구하는 흐름

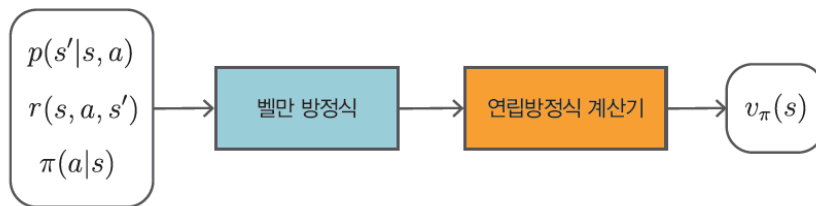


그림 1. 앞 장에서 한 일. 환경 모델 (p, r) 과 정책 π 를 벨만 방정식에 넣고 가운데 주황 상자 “연립방정식 계산기”로 가치 함수를 얻었다. 이 장에서는 그 상자를 반복 계산으로 바꾼다. 출처: 원서 그림 4-1.

이 장이 바꾸는 것은 그림 1의 가운데 상자 하나다. 착상은 한 줄로 요약된다. 식 (1)의 등호를 대입으로 바꿔 읽는다. 우변에 현재의 추정치 V_k 를 넣고 좌변을 새 추정치 V_{k+1} 로 삼는 것이다.

$$V_{k+1}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma V_k(s') \}. \quad (2)$$

참값 v_{π} 가 아니라 추정치임을 나타내기 위해 대문자 V 를 쓴다. 이 대입을 되풀이하면 아무 값에서 출발해도 참값 v_{π} 하나로 수렴하는데, 그 까닭은 갱신식을 구체적인 숫자로 적어 놓은 §4.1.1 끝에서 따진다.

이 장의 목표는 이 갱신을 빠르게 삼아 두 알고리즘을 세우는 것이다. 정책 반복법은 평가와 개선을 번갈아 반복하고, 가치 반복법은 둘을 하나로 합쳐 최적 가치를 직접 구한다. 두 알고리즘 모두 환경 모델 p 와 r 를 완전히 알고 있다고 전제하며, 이 전제가 깨지는 것이 다음 장부터다.

본고의 구성과 기여

본고는 원서 4장의 목차를 그대로 따른다. §4.1에서 벨만 방정식의 등호를 갱신식으로 바꿔 읽고 그 반복을 두 칸짜리 그리드 월드에 적용하며 두 가지 구현을 비교한다. §4.2에서 3×4 그리드 월드로 무대를 옮기고, §4.3에서 평가에 개선을 붙여 정책 반복법을 세운 뒤 §4.4에서 그것을 구현한다. §4.5에서 평가를 한 스윕으로 줄여 개선과 합친 가치 반복법을 얻으며, §4.6이 정리다.

원서 4장은 예제 코드를 제공하므로 본고는 그것을 그대로 쓰되, 원서가 관찰로만 남기거나 아예 짚지 않은 다섯 자리에 논의를 더하였다. 절을 따로 만들어 밀어 두지 않고, 그것이 필요해지는 자리의 본문에 “[보충]”으로 표시해 두었다.

1. **동적 프로그래밍이라는 이름**(§4.1.1 끝). 원서는 이 이름을 풀지 않고 쓴다. 벨만이 이 이름을 고른 사정과 programming의 당시 뜻을 밝히고, 기법으로서의 정의인 최적 부분 구조와 중복되는 부분 문제에 반복적 정책 평가를 하나씩 대응시킨다. 교과서 동적 계획법이 표를 한 번 채우고 끝나는 것과 달리 여기서 되풀이가 필요한 까닭이 부분 문제 의존 관계의 순환에 있음도 함께 적는다.
2. **제자리 갱신이 빠른 이유**(§4.1 끝). 원서는 배열 하나를 쓰는 구현이 76스윕에서 60스윕으로 줄어든다는 사실을 관찰로 제시한다. 본고는 이것이 수치해석의 야코비 반복과 가우스-자이델 반복의 차이라는 이름을 붙이고, 정지 문턱을 낮출수록 그 이점이 커진다는 것과 그 대가가 무엇인지를 함께 적는다.
3. **정지 문턱이 남기는 오차**(§4.1 끝). $\delta_k < \varepsilon$ 에서 멈춘 값이 참값과 얼마나 떨어져 있는지를 따진다. 오차는 문턱 자체가 아니라 그 $\gamma/(1-\gamma)$ 배까지 허용되며, 앞 장의 정확해 표와 이 장의 반복 계산 표가 세 칸에서 갈리는 것이 그 실물이다.
4. **두 알고리즘의 실제 비용**(§4.5 끝). 원서의 비교표는 정책 반복법을 “사이클 5회”, 가치 반복법을 “스윕 4회”로 적는데, 단위가 달라 그대로 견줄 수 없다. 같은 단위로 환산하면 32 스윕 대 4 스윕, 백업 횟수로는 1776회 대 240회다.
5. **최적 가치의 닫힌 형태와 유한 수렴**(§4.5 끝). $v_*(s) = \gamma^{d(s)-1}$ (d 는 폭탄 칸을 지나지 않는 최단 걸음 수)임을 보이고, 그로부터 가치 반복법이 문턱과 무관하게 유한 스윕 안에 끝남을 설명한다. 제자리 갱신은 3스윕, 동기 갱신은 5스윕이 걸린다.

거리가 매번 줄어든다는 사실에서 수렴을 이끌어내는 형식적 증명, 반복이 오차를 줄이는 속도를 정확히 재는 계산, 닫힌 형태의 경우 분석 같은 대목은 이 장의 줄거리를 따라가는 데 꼭 필요하지 않아 덜어내고 결과와 그 뜻만 남겼다.

4.1 동적 프로그래밍과 정책 평가

4.1.1 동적 프로그래밍 기초

소재는 앞 장에서 쓴 두 칸짜리 그리드 월드다(그림 2).

식 (2)을 이 환경에 맞게 두 단계로 간추린다. 먼저 하나로 묶여 있던 $\sum_{a,s'}$ 를 $\sum_a \sum_{s'}$ 로 나누고, 다음으로 상태 전이가 결정적이라는 사실을 쓴다. 이 환경에서는 상태와 행동이 정해지면 다음 상태가 $s' = f(s, a)$ 하나로 정해지므로, s' 에 대한 합은 항이 하나뿐이라 통째로 사라진다. 그림 3이 그 두 단계다.

남는 것은 다음 갱신식이다.

$$V_{k+1}(s) = \sum_a \pi(a | s) \{ r(s, a, f(s, a)) + \gamma V_k(f(s, a)) \}. \quad (3)$$

이 갱신을 반복하는 것을 반복적 정책 평가(iterative policy evaluation)라 한다.

그림 4-2 두 칸짜리 그리드 월드(L1에서 L2로 이동하면 +1 보상, 벽에 부딪히면 -1 보상)

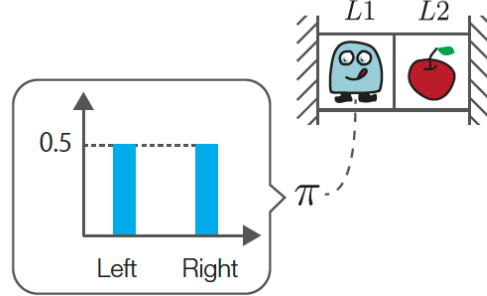


그림 2. 두 칸짜리 그리드 월드. L1에서 L2로 이동하면 +1, 벽에 부딪히면 -1이다. 평가할 정책은 두 칸 어디에서나 Left와 Right를 0.5씩 고르는 무작위 정책이며 $\gamma = 0.9$ 다. 출처: 원서 그림 4-2.

그림 4-3 상태 전이가 결정적인 경우의 갱신식

$$\begin{aligned}
 & V_{k+1}(s) = \sum_{a,s'} \pi(a|s) p(s'|s,a) \{r(s,a,s') + \gamma V_k(s')\} \quad [\text{식 4.2}] \\
 & \quad \downarrow \sum \text{를 둘로 구분하여 표기} \\
 & V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) \{r(s,a,s') + \gamma V_k(s')\} \\
 & \quad \downarrow \text{상태 전이가 결정적} \\
 & \quad s' = f(s,a) \text{ 일 때} \\
 & V_{k+1}(s) = \sum_a \pi(a|s) \{r(s,a,s') + \gamma V_k(s')\} \quad [\text{식 4.3}]
 \end{aligned}$$

그림 3. 위에서 아래로 내려오며 $\sum$ 이 하나씩 사라진다. 1단계는 $\sum_{a,s'}$ 를 $\sum_a \sum_{s'}$ 로 분리하는 표기 정리이고, 2단계에서 결정적 전이를 쓰면 $\sum_{s'}$ 가 없어진다. 맨 아래가 식 (3)이며 구현에 실제로 쓰는 형태다. 출처: 원서 그림 4-3.

그림 4-4 상태 L1부터 시작하는 백업 다이어그램

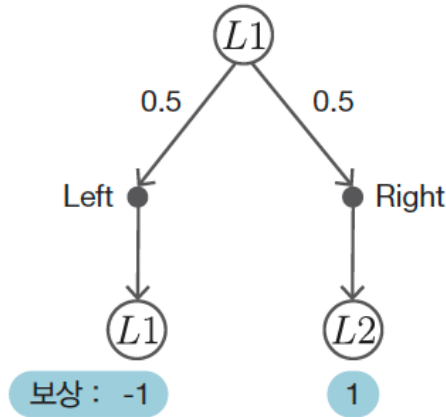


그림 4-5 가치 함수 1차 갱신

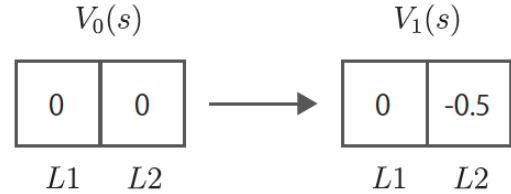


그림 4. L1의 갱신을 그린 백업 다이어그램(왼쪽)과 $V_0 = (0, 0)$ 에서 한 번 갱신한 결과(오른쪽). $V_1(L1)$ 이 여전히 0인 것은 -1과 +1이 반반이라 상쇄되기 때문이고, $V_1(L2) = 0.5 \times 0 + 0.5 \times (-1) = -0.5$ 다. 참값 $(-2.25, -2.75)$ 와는 아직 멀다. 출처: 원서 그림 4-4, 4-5.

그림 4의 갱신을 식으로 쓰면 다음과 같다.

$$V_{k+1}(L1) = 0.5\{-1 + 0.9 V_k(L1)\} + 0.5\{1 + 0.9 V_k(L2)\}, \quad (4)$$

$$V_{k+1}(L2) = 0.5\{0 + 0.9 V_k(L1)\} + 0.5\{-1 + 0.9 V_k(L2)\}. \quad (5)$$

왜 수렴하는가. 이 되풀이가 참값에 이르는 근거는 둘이다.

하나는 참값이 이 갱신의 고정점이라는 것이다. 식 (4)과 식 (5)의 우변에 앞 장에서 구한 참값 $(-2.25, -2.75)$ 를 넣어 보면 $0.5\{-1 + 0.9 \times (-2.25)\} + 0.5\{1 + 0.9 \times (-2.75)\} = -2.25$ 이고 L2 쪽도 마찬가지로 -2.75 가 나온다. 넣은 값이 그대로 다시 나온다. 애초에 벨만 방정식이 성립한다는 말이 이 뜻이었다. 넣은 것이 그대로 나오는 값을 그 갱신의 고정점이라 하며, 앞 장[2]에서 $x = \cos x$ 를 예로 든 그것이다. 계산기에 아무 수나 넣고 cos을 계속 누르면 0.739...로 빨려 드는데, 그 값이 cos의 고정점이다. 다른 점은 여기서 다루는 것이 숫자 하나가 아니라 상태마다 값이 하나씩 적힌 표라는 것뿐이다.

다른 하나는 이 갱신이 두 표 사이의 거리를 줄인다는 것이다. 서로 다른 두 표를 넣어 견주면 알 수 있다. 표가 들어가는 자리는 γV_k 하나뿐이고 그 앞에 붙은 $\sum_a \pi(a | s)$ 는 합이 1인 확률로 평균을 내는 일이므로, 두 표의 차이를 아무리 섞어 평균 내도 그중 가장 큰 차이보다 커질 수 없다. 남는 것은 앞의 γ 뿐이다. 즉 한 번 대입할 때마다 두 표의 거리는 γ 배 이하로 줄어든다.

둘을 합치면 결론이 나온다. 두 표 중 하나를 참값 v_π 로 잡으면 그것은 고정점이라 대입해도 제자리에 있으므로, 줄어드는 “두 표의 거리”가 곧 “참값과의 오차”다. 그래서 아무 표에서 출발해도 k 번 대입한 뒤의 오차는 γ^k 배 이하로 눌러 0으로 간다. 고정점이 둘일 수도 없다. 서로 다른 고정점이 둘 있다면 대입해도 둘 다 제자리이므로 그 거리가 그대로여야 하는데, 대입은 거리를 반드시 γ 배 이하로 줄이므로 거리는 0일 수밖에 없다. 두 근거 중 뒤의 것을 떠받치는 것이 $\gamma < 1$ 이며, 이 장이 다루는 세 알고리즘이 모두 여기에 기댄다.

언제 멈추는가. 참값을 모르므로 “더 이상 거의 변하지 않는다”를 수렴의 신호로 삼는다. 한 번의 갱신에서 가장 크게 변한 양을

$$\delta_k = \|V_k - V_{k-1}\|_\infty = \max_s |V_k(s) - V_{k-1}(s)| \quad (6)$$

로 두고 $\delta_k < \varepsilon$ 이면 종료한다. 이 규칙이 실제로 얼마의 오차를 남기는지는 이 절 끝의 [보충] 정지 문턱이 남기는 오차에서 따로 따진다.

[보충] 왜 “동적 프로그래밍”인가

여기까지 한 일에는 “동적”인 것도 “프로그래밍”인 것도 나오지 않았다. 이름과 내용이 어긋나 보이는 데에는 까닭이 있다.

이름의 유래. 이 이름은 기법의 성질을 담으려고 지은 것이 아니다. 벨만이 1950년대 RAND 연구소에서 붙였는데, 그 사정을 자서전[5]에 적어 두었다. 당시 국방장관이던 월슨이 “research”나 “mathematics”라는 말을 몹시 싫어하여, 예산을 얻으려면 수학 연구로 보이지 않는 이름이 필요했다는 것이다. programming은 컴퓨터 프로그램을 짜는 일이 아니라 그 무렵 운용과학에서 쓰던 말로 계획을 세우고 일정을 짜는 일을 뜻했다. 같은 시기의 *linear programming*(선형계획법)이 그 용법이며, 그 관례를 따르면 이 말도 동적계획법으로 옮기는 편이 원뜻에 가깝다. dynamic은 “정적”의 반대말로, 결정이 한 번에 끝나지 않고 여러 단계에 걸쳐 이어지는 문제라는 뜻이다. 둘을 합친 원뜻은 다단계 의사결정 계획법에 가깝다. 요컨대 이름에서 기법을 읽어내려 애쓸 일이 아니다.

기법으로서의 정의. 내용 쪽의 정의는 분명하다. 동적 프로그래밍은 다음 두 성질을 가진 문제를, 부분 문제의 답을 표에 적어 두었다가 다시 쓰는 방식으로 푸는 방법이다. 하나는 최적 부분 구조로, 큰 문제의 답이 부분 문제의 답으로 표현되는 것이다. 다른 하나는 중복되는 부분 문제로, 같은 부분 문제가 여러 번 나타나 한 번 풀어 적어 두면 이득이 되는 것이다. 뒤의 것이 없으면 표를 둘 이유가 없어 분할 정복과 다르지 않다.

반복적 정책 평가와의 대응. 앞에서 세운 것이 이 정의에 그대로 들어맞는다. 최적 부분 구조에 해당하는 것은 벨만 방정식 자체다. 식 (1)은 $v_{\pi}(s)$ 를 $v_{\pi}(s')$ 으로 적은 식이며, 이는 “이 상태의 답은 다음 상태의 답으로 만들어진다”는 진술이다. 중복되는 부분 문제에 해당하는 것은 여러 상태가 같은 s' 을 공유한다는 사실이다. 식 (4)과 식 (5)에서 $V_k(L1)$ 은 양쪽 모두에 나타난다. 그리고 표에 해당하는 것이 V 배열이다. 한 스왑에서 $V_k(s')$ 을 읽어 쓰는 일이 곧 적어 둔 부분 문제의 답을 다시 쓰는 일이다. 표가 없다면 한 상태의 값을 얻기 위해 미래의 경로를 매번 끝까지 펼쳐야 하고, 경로 수는 걸음마다 곱으로 늘어난다.

왜 표를 한 번에 채우지 못하는가. 그런데 교과서에서 동적 계획법이라 부르는 것들, 이를테면 피보나치 수나 배낭 문제는 표를 한 번 채우면 끝난다. 여기서는 `while`로 되풀이한다. 차이는 부분 문제 사이의 의존 관계에 있다. 교과서의 예들은 그 관계가 순환하지 않아, 아무것에도 기대지 않는 것부터 차례로 확정해 나가는 순서가 존재한다. 반면 식 (4)과 식 (5)을 보면 $V(L1)$ 이 $V(L2)$ 에 기대고 $V(L2)$ 가 다시 $V(L1)$ 에 기대다. 먼저 확정할 수 있는 것이 없으므로 그런 순서가 아예 없다. 그래서 순서를 정하는 대신 아무 값에서 시작해 고정점으로 밀어 넣으며, 그것이 되는 근거가 바로 앞에서 따진 γ 배 축소다. 되풀이는 이 문제가 어려워서가 아니라 의존 관계에 고리가 있어 치르는 값이다. 덧붙이면 과제가 유한한 길이에서 끝나고 시간 단계 t 를 상태에 넣으면, t 는 늘 커지는 쪽으로만 움직이므로 고리가 끊긴다. 그때는 뒤에서 앞으로 한 번 훑으면 끝나 교과서의 모습과 같아진다.

강화 학습에서의 좁은 뜻. 서튼과 바르토[3]는 이 말을 더 좁게 쓴다. 환경의 완벽한 모델이 마르코프 결정 과정으로 주어졌을 때 최적 정책을 구하는 알고리즘들의 모음이라는 것이다. 모델을 안다는 전제가 정의 안에 이미 들어 있다. 그래서 이 장의 방법들은 엄밀히는 학습이 아니라 계획(planning)에 속하며, 그 전제가 깨지는 다음 장부터가 학습이다.

4.1.2 반복적 정책 평가: 첫 번째 구현

식 (4), (5)을 코드로 옮기는 방법은 두 가지다. 첫 번째는 식을 글자 그대로 옮겨 옛 배열 V 를 읽고 새 배열 new_V 에 쓰는 것이다.

그림 5이 이 방식의 갱신을 그린 것이다.

```
V = {'L1': 0.0, 'L2': 0.0}
new_V = V.copy()
while True:
    new_V['L1'] = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    new_V['L2'] = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2'])
    delta = max(abs(new_V['L1'] - V['L1']), abs(new_V['L2'] - V['L2']))
    V = new_V.copy() # 한 스왑이 끝난 뒤에 통째로 교체
    if delta < 0.0001:
        break
```

코드 1. 배열 두 개를 쓰는 구현 (s01_dp.py)

그림 4-6 4.1.2절의 갱신 방법

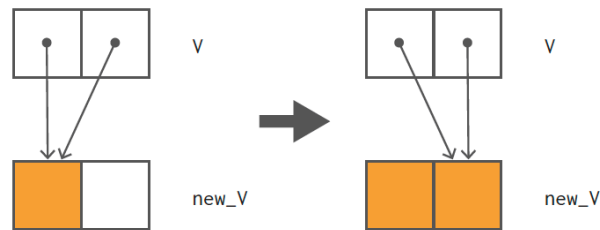


그림 5. 배열 두 개를 쓰는 구현의 갱신 방식. 화살표의 출발점이 항상 위쪽 V 배열이므로 한 스왑 동안 V 는 고정된다. $L1$ 을 갱신해도 $L2$ 의 계산에는 여전히 옛 값이 쓰인다. 출처: 원서 그림 4-6.

이 구현은 76스왑 만에 멈추며 $(-2.249168, -2.749168)$ 을 내놓는다. 앞 장에서 연립방정식으로 얻은 정확해 $(-2.25, -2.75)$ 와 소수점 셋째 자리까지 같다. 남은 오차 8.3×10^{-4} 는 문턱 10^{-4} 에서 멈춘 대가이며, 그 크기가 왜 하필 문턱의 아홉 배인지는 이 절 끝의 [보충]에서 따진다.

4.1.3 반복적 정책 평가: 다른 구현 방법

다른 하나는 배열 하나만 두고 계산하는 즉시 덮어쓰는 것이다. 그림 6에서 보듯 new_V 가 사라지고 화살표가 같은 배열 안에서 돌아온다.

```
V = {'L1': 0.0, 'L2': 0.0}
while True:
    t = 0.5 * (-1 + 0.9 * V['L1']) + 0.5 * (1 + 0.9 * V['L2'])
    delta = abs(t - V['L1']); V['L1'] = t # 여기서 곧바로 덮어씀
    t = 0.5 * (0 + 0.9 * V['L1']) + 0.5 * (-1 + 0.9 * V['L2']) # 방금 바뀐 값을 씀
    delta = max(delta, abs(t - V['L2'])); V['L2'] = t
    if delta < 0.0001:
        break
```

코드 2. 제자리 갱신 구현 (s02_dp_inplace.py)

두 구현은 같은 답에 이르지만 걸리는 스왑 수가 다르다. 표 1이 실행 결과다.

제자리 갱신이 76회에서 60회로 줄었고 배열도 하나만 쓴다. 원서는 “새 값이 곧바로 다음 계산에 반영되기 때문”이라는 설명에서 멈추는데, 다음 절에서 이 현상에 이름과 숫자를 붙인다.

[보충] 제자리 갱신이 빠른 이유: 야코비와 가우스-자이델

두 구현의 차이는 강화 학습에 고유한 것이 아니라 선형 방정식을 반복법으로 푸는 고전적인 두 방식의 차이다. 배열을 둘 쓰는 구현은 한 스왑 동안 모든 상태가 같은 옛 표만 참조한다. 이것을

그림 4-7 새로운 갱신 방식

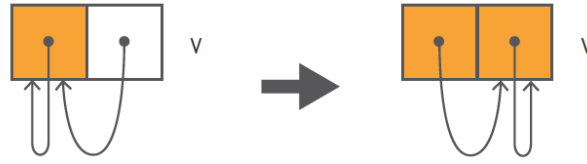


그림 6. 제자리 갱신의 갱신 방식. `new_v`가 없고 화살표가 같은 배열 안에서 돌아온다. L2를 갱신할 때 방금 새로 계산된 L1을 쓰므로 갱신된 정보가 같은 스왑 안에서 전파된다. 출처: 원서 그림 4-7.

표 1. 두 구현의 실행 결과($\varepsilon = 10^{-4}$, $\gamma = 0.9$). 참값은 $(-2.25, -2.75)$ 다. 마지막 열의 뜻은 이 절 끝의 [보충] 정지 문턱이 남기는 오차에서 다룬다.

구현	결과 (L1, L2)	스왑	최종 δ	실제 오차	상한 $\gamma\delta/(1-\gamma)$
배열 둘 (s01)	$(-2.249168, -2.749168)$	76	9.250×10^{-5}	8.325×10^{-4}	8.325×10^{-4}
제자리 (s02)	$(-2.249378, -2.749420)$	60	9.320×10^{-5}	6.218×10^{-4}	8.388×10^{-4}

야코비 반복이라 한다. 제자리 갱신은 s 번째 상태를 계산할 때 이미 갱신이 끝난 $1, \dots, s-1$ 번째 값을 쓴다. 이것이 가우스-자이델 반복이며, 코드에서 `v['L1']`을 덮어쓴 뒤 그 값을 곧바로 `v['L2']` 계산에 쓰는 두 줄이 그 차이이다.

새 값이 같은 스왑 안에서 곧바로 퍼지므로 정보가 한 스왑에 더 멀리 나아가고, 그만큼 스왑 수가 줄어든다. 줄어드는 정도는 정지 문턱에 따라 달라진다. 표 2가 실측이다.

표 2. 정지 문턱에 따른 두 구현의 스왑 수. 문턱을 낮출수록 실측 비가 0.75 부근으로 내려간다. $\varepsilon = 10^{-4}$ 행이 표 1의 76과 60이다.

ε	배열 둘(야코비)	제자리(가우스-자이델)	실측 비
10^{-2}	32	27	0.8438
10^{-3}	54	44	0.8148
10^{-4}	76	60	0.7895
10^{-6}	119	93	0.7815
10^{-8}	163	126	0.7730

문턱이 클 때 실측 비가 더 큰 것은 초기 몇 스왑의 과도 구간이 전체에서 차지하는 비중이 크기 때문이다. 문턱을 낮출수록 그 비중이 줄어 두 방식의 본래 속도 차이가 드러난다. 그림 7 왼쪽이 이 관계를 로그 눈금에서 직접 보여준다. 두 곡선 모두 직선이고 제자리 갱신 쪽 기울기가 조금 더 가파르다.

가우스-자이델의 이점이 공짜는 아니라는 점도 적어 둘 만하다. 제자리 갱신은 “이미 갱신이 끝난 상태”가 무엇인지에 기대므로 스왑 순서를 바꾸면 속도도 바뀐다. 또 방금 쓴 값을 곧바로 읽어야 하므로 상태들을 병렬로 갱신할 수 없다. 야코비 갱신은 순서에 무관하고 완전히 병렬화되지만 배열이 둘 필요하다. 여기서 얻을 결론은 “제자리가 항상 낫다”가 아니라 “76 → 60은 우연이 아니라 두 방식의 구조적 차이”라는 것이다.

[보충] 정지 문턱이 남기는 오차

$\delta_k < \varepsilon$ 에서 멈추면 참값과 얼마나 떨어져 있는가. δ_k 는 참값과의 거리가 아니라 직전 추정치와의 거리이므로 이 물음은 자명하지 않다. 답은 다음과 같다.

$$\|V_k - v_\pi\|_\infty \leq \frac{\gamma}{1-\gamma} \|V_k - V_{k-1}\|_\infty = \frac{\gamma}{1-\gamma} \delta_k. \quad (7)$$

근거로 쓰는 것은 앞 장에서 본 두 가지뿐이다. 하나는 한 번 대입할 때마다 참값과의 거리가 γ 배 이하로 줄어든다는 것이고, 다른 하나는 “직전 추정치와 참값의 거리”를 “직전과 지금의 거리 δ_k ”

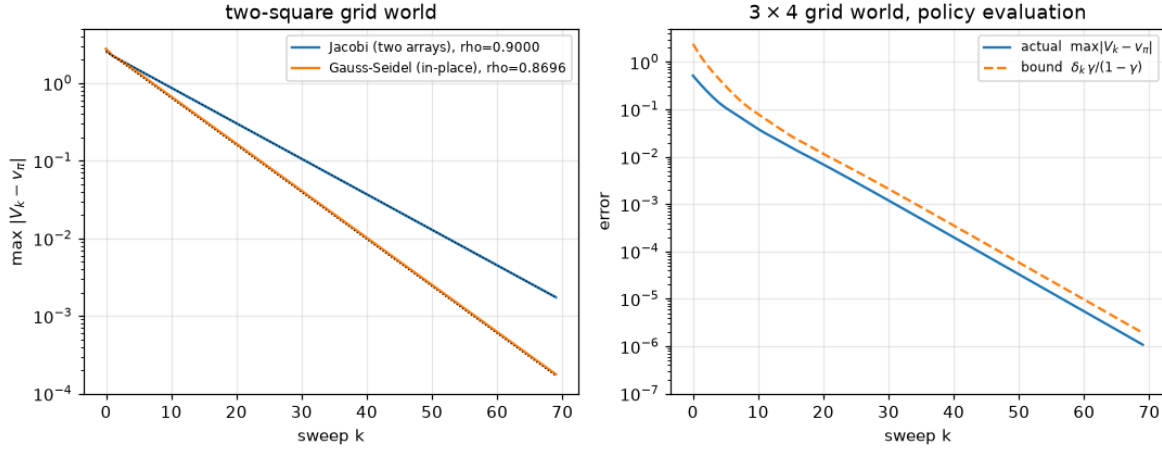


그림 7. 왼쪽: 두 칸 세계에서 야코비(배열 둘)와 가우스-자이델(제자리) 갱신의 오차(로그 눈금). 점선은 각 방식의 점근 수렴 속도가 예측하는 직선으로, 곡선과 겹쳐 구별되지 않는다. 오른쪽: 3×4 그리드의 반복적 정책 평가에서 실제 오차와 식 (7)의 상한. 상한이 언제나 위에 있고 두 선이 나란하다. `ch04/paper/fig_convergence.py`에서 생성.

더하기 “지금의 오차”로 쪼갤 수 있다는 것이다. 둘을 합치면 $(1 - \gamma)\|V_k - v_\pi\|_\infty \leq \gamma \delta_k$ 가 된다. $\gamma = 0.9$ 이면 계수가 $\gamma/(1 - \gamma) = 9$ 이므로, 문턱이 10^{-4} 라도 오차는 그 아홉 배까지 허용된다. 표 1의 배열 둘 구현이 정확히 그 경계에 있다. 실제 오차 8.325×10^{-4} 가 상한 $9 \times 9.250 \times 10^{-5} = 8.325 \times 10^{-4}$ 과 유효숫자 네 자리까지 같다. 원서 4.1.2절이 “오차 $\approx \delta \times 9$ ”라고 적은 것은 이 환경의 배열 둘 구현에서는 근사가 아니라 사실상 등식인 셈이다. 반면 제자리 갱신의 오차 6.218×10^{-4} 는 상한 8.388×10^{-4} 보다 작다. 두 구현이 남기는 오차의 모양이 서로 다르기 때문이다.

앞 장의 표와 이 장의 표는 왜 다른가. 이 상한은 두 장을 잇는 곳에서 눈에 보이는 차이를 만든다. 앞 장은 3×4 그리드의 무작위 정책 가치를 연립방정식으로 정확히 풀었고, 이 장은 같은 값을 문턱 10^{-3} 의 반복 계산으로 구한다. 두 결과를 소수 둘째 자리로 반올림하면 세 칸에서 값이 다르다.

표 3. 같은 문제, 다른 표. 앞 장의 정확해와 이 장의 반복 계산($\epsilon = 10^{-3}$, 23스weep)이 세 칸에서 소수 둘째 자리까지 같린다. 차이는 모두 실제 오차 4.877×10^{-3} 안에 있다.

상태	앞 장(정확해)	이 장(문턱 10^{-3})	차이
(0, 1)	0.09	0.10	+0.0032
(2, 2)	-0.44	-0.43	+0.0025
(2, 3)	-0.79	-0.78	+0.0019

표 3의 차이는 모두 양수다. $V_0 = 0$ 에서 출발했고 이 문제의 참값이 대부분 음수이므로 추정치가 위에서 아래로 내려오는 중이기 때문이다. 표 4가 문턱을 낮출 때 이 오차가 어떻게 줄어드는지 보여준다.

마지막 행이 앞 장에서 인용한 값이다. 앞 장은 이 장의 반복 계산과 연립방정식의 해가 최대 오차 4.644×10^{-10} 로 일치한다고 적었는데, 표 4는 그 숫자가 어디서 나왔는지를 보여준다. 문턱 10^{-10} 에서 113스weep을 돌린 결과이며, 두 방법이 같은 방정식을 푼다는 증거인 동시에 반복법의 답이 언제나 문턱만큼 흐릿하다는 사실의 확인이기도 하다. 그림 7 오른쪽이 스weep마다의 실제 오차와 상한을 나란히 그린 것이다. 두 선이 나란한 직선이며 상한이 언제나 위에 있다.

표 4. 3×4 그리드의 반복적 정책 평가에서 문턱, 스윙 수, 오차의 관계($\gamma = 0.9$). 실제 오차가 모든 행에서 식 (7)의 상한 아래에 있고 대략 그 0.6배다.

ϵ	스윙	최종 δ	실제 오차	상한 $\gamma\delta/(1-\gamma)$
10^{-2}	11	8.818×10^{-3}	3.858×10^{-2}	7.936×10^{-2}
10^{-3}	23	9.152×10^{-4}	4.877×10^{-3}	8.237×10^{-3}
10^{-4}	36	9.587×10^{-5}	4.888×10^{-4}	8.628×10^{-4}
10^{-6}	62	8.973×10^{-7}	4.545×10^{-6}	8.075×10^{-6}
10^{-8}	87	9.927×10^{-9}	5.028×10^{-8}	8.934×10^{-8}
10^{-10}	113	9.169×10^{-11}	4.644×10^{-10}	8.253×10^{-10}

4.2 더 큰 문제를 향해

4.2.1 GridWorld 클래스 구현

두 칸 세계는 손으로도 풀린다. 이제 12칸짜리 문제로 옮겨 간다. 이 환경은 이후 5, 6장에서도 계속 쓰인다.

그림 4-8 3×4 그리드 월드

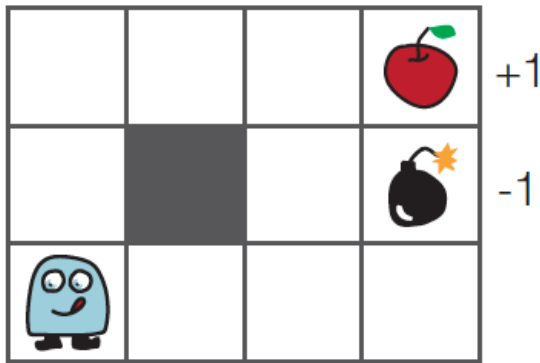


그림 4-9 보상 맵의 좌표계

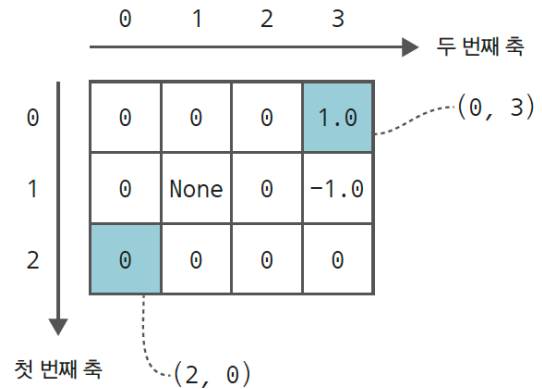


그림 8. 3×4 그리드 월드(왼쪽)와 좌표 약속(오른쪽). 오른쪽 위 사과(+1)가 목표 상태 (0,3), 그 아래 폭탄(-1)이 (1,3), 가운데 회색 칸이 벽 (1,1)이며 출발은 (2,0)이다. 사과에 도달하면 에피소드가 끝나므로 일회성 과제다. 출처: 원서 그림 4-8, 4-9.

환경은 `common/gridworld.py`의 `GridWorld` 클래스가 담당한다. 좌표 약속과 보상 맵이 그림 8을 그대로 옮긴 것이며, 벽 자리는 보상 맵에서 `None`으로 표시한다.

```
class GridWorld:
    def __init__(self):
        self.action_space = [0, 1, 2, 3]
        self.action_meaning = {0: "UP", 1: "DOWN", 2: "LEFT", 3: "RIGHT"}
        self.reward_map = np.array(
            [[0, 0, 0, 1.0],
             [0, None, 0, -1.0],
             [0, 0, 0, 0]]
        )
        self.goal_state = (0, 3)
        self.wall_state = (1, 1)
        self.start_state = (2, 0)
```

코드 3. 환경 클래스의 뼈대 (`common/gridworld.py`)

이 장에서 실제로 쓰는 메서드는 표 5의 다섯 개다.

표 5. 이 장에서 쓰는 GridWorld의 메서드. 앞의 네 개가 환경 모델 p 와 r 를 드러내는 통로이며, 이 통로에 직접 접근할 수 있다는 것이 이 장의 전제다.

메서드	하는 일
<code>states()</code>	모든 상태를 (h, w) 로 하나씩 내놓는 제너레이터
<code>actions()</code>	$[0, 1, 2, 3]$
<code>next_state(s, a)</code>	상태 전이. 벽이나 테두리 밖이면 제자리
<code>reward(s, a, s2)</code>	보상. 도착 칸만 보고 정한다(<code>reward_map[s2]</code>)
<code>render_v(V, pi)</code>	가치 함수와 정책을 색과 화살표로 그린다

보상 설계에 주의할 점이 하나 있다. `reward()`가 도착 칸 s' 만 보고 값을 정하므로 목표와 폭탄 칸에 들어갈 때 보상이 생기고 나머지 칸은 모두 0이다. 즉 “빨리 가라”고 재촉하는 장치가 환경에 없다. 그 역할을 대신하는 것이 할인율이다. 늦게 받는 $+1$ 은 γ^k 만큼 깎이므로 최단 경로가 저절로 유리해진다. §4.5 끝의 [보충]에서 보듯 이 환경의 최적 가치가 정확히 γ 의 거듭제곱으로 나오는 것이 그 귀결이다.

`render_v()`의 출력을 먼저 확인해 두면 이후의 결과를 읽기 쉽다. `s03_gridworld_play.py`가 더미 값을 넣어 시각화만 확인하는 예제이고, 그림 9이 그 출력이다.

그림 4-10 3×4 그리드 월드 시각화

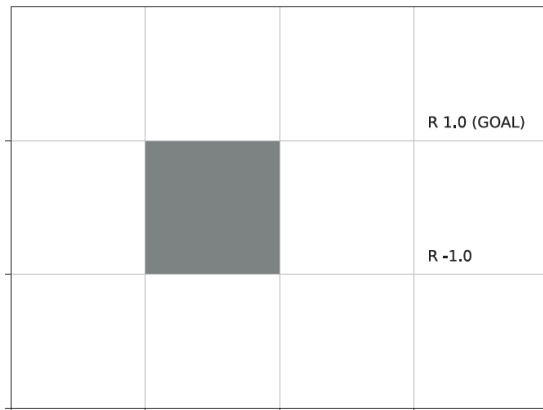


그림 4-11 3×4 그리드 월드 시각화(상태 가치 함수 입력)



그림 9. `render_v()`에 아무것도 넘기지 않았을 때(왼쪽)와 `np.random.randn()`으로 만든 의미 없는 값을 넣었을 때(오른쪽). 값이 클수록 초록, 작을수록 빨강이며 색이 곧 가치의 크기다. 회색 칸(벽)에는 값이 표시되지 않는다. 출처: 원서 그림 4-10, 4-11.

4.2.2 defaultdict 사용법

상태가 (h, w) 튜플이므로 가치 함수와 정책은 딕셔너리로 다루는 편이 자연스럽다. 다만 없는 키에 접근하면 `KeyError`가 나므로 `collections.defaultdict`를 쓴다.

```
from collections import defaultdict

V = defaultdict(lambda: 0) # 없는 상태의 가치는 0
pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}) # 없는 상태는 무작위 정책
```

코드 4. 기본값이 있는 딕셔너리

모든 상태를 미리 0으로 채워 두는 초기화 코드가 필요 없다는 것이 이점이다. `v[(0, 0)]`처럼 처음 접근하는 순간 기본값이 만들어지고, 정책도 상태를 일일이 나열하지 않고 “기본은 무작위”를 표현할 수 있다.

주의할 점은 `V.keys()`가 한 번이라도 접근한 상태만 담는다는 것이다. 그래서 코드 5의 `policy_eval()`은 `eval_onestep()`이 `env.states()`로 전체를 한 번 훑은 뒤에야 `V.keys()`로 δ 를 잰다. 순서를 바꾸면 첫 스왑에서 아직 만들어지지 않은 상태가 빠진 채 δ 가 계산된다.

4.2.3 반복적 정책 평가 구현

스왑 순서

그림 4-12 각 상태에 순차적으로 접근하기

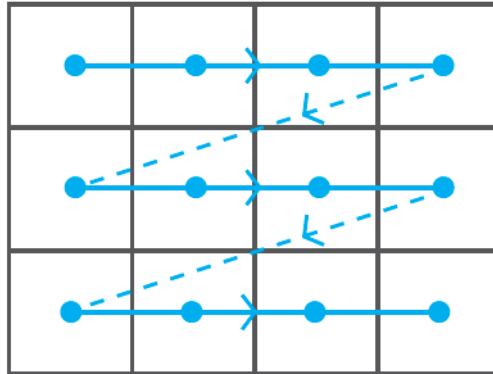


그림 10. 한 번의 스왑에서 상태를 훑는 순서. 왼쪽 위에서 오른쪽으로, 줄이 끝나면 다음 줄로 내려간다. 실선이 오른쪽으로의 이동, 점선이 줄바꿈이며 `env.states()`가 내놓는 순서 그대로다. 이 순서가 제자리 갱신에서 정보가 전파되는 방향을 정한다. 목표가 오른쪽 위에 있으므로 정보가 왼쪽 아래로 되돌아오는 데는 여러 스왑이 필요하다. 출처: 원서 그림 4-12.

구현

구현은 식 (3)을 그대로 옮긴 것이다. `eval_onestep()`이 한 번의 스왑에 해당하고, `policy_eval()`이 그 반복에 해당한다.

```
def eval_onestep(pi, V, env, gamma=0.9):
    for state in env.states():
        if state == env.goal_state:      # 목표는 종료 상태이므로 v(goal) = 0
            V[state] = 0
            continue
        new_V = 0
        for action, action_prob in pi[state].items():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            new_V += action_prob * (r + gamma * V[next_state])
        V[state] = new_V                # 제자리 갱신
    return V

def policy_eval(pi, V, env, gamma, threshold=0.001):
    while True:
        old_V = V.copy()
        V = eval_onestep(pi, V, env, gamma)
        delta = max(abs(V[s] - old_V[s]) for s in V.keys())
        if delta < threshold:
            break
```

```
return V
```

코드 5. 반복적 정책 평가 (s04_policy_eval.py)

목표 상태를 0으로 고정하는 처리를 빼먹으면 목표에서 다시 출발하는 것처럼 계산되어 값이 발산한다. 종료 상태의 가치가 0이라는 것은 규약이 아니라 정의다. 그 뒤로 받을 보상이 없기 때문이다.

무작위 정책의 평가

무작위 정책 $\pi(a | s) = 0.25$ 를 문턱 10^{-3} 로 평가하면 23스텝 만에 멈춘다.

그림 4-13 무작위 정책의 가치 함수

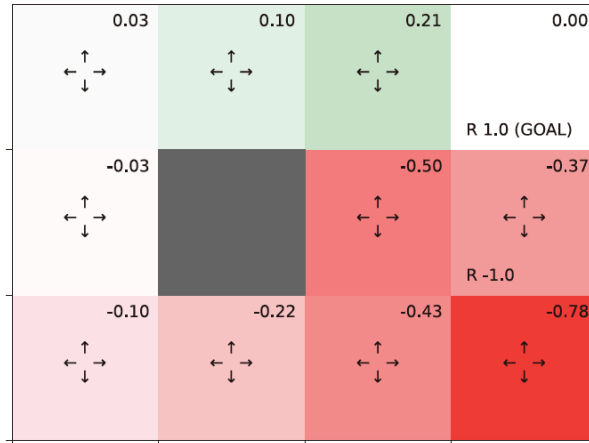


그림 11. 무작위 정책의 가치 함수. 각 칸의 화살표 네 개가 무작위 정책을 나타낸다. 목표 왼쪽 칸(0.21)만 초록이고 오른쪽 아래로 갈수록 빨강이 짙어진다. 가장 나쁜 (2, 3)의 -0.78 은 폭탄 바로 아래라 무작위로 걸으면 폭탄에 닿기 쉬운 자리이고, 왼쪽 아래 (2, 0)의 -0.10 은 폭탄에서 멀어 덜 나쁘다. 출처: 원서 그림 4-13.

이 값들은 무작위로 걸었을 때의 성적이므로 대부분 음수다. 좋은 정책을 찾으면 전부 양수가 되며, 그것이 다음 절의 목표다. 앞 장의 정확해와 이 표가 세 칸에서 다른 이유는 §4.1 끝의 [보충]에서 이미 다루었다.

4.3 정책 반복법

4.3.1 정책 개선

지금까지는 주어진 정책을 평가하기만 했다. 이제 정책을 더 좋게 만드는 단계를 붙인다. 앞 장에서 최적 가치 v_* 를 알면 최적 정책이 탐욕적으로 뽑힌다는 것을 보았는데, 우리는 아직 v_* 를 모른다. 대신 지금 정책의 가치 v_μ 는 방금 구했다. 그 자리에 v_μ 를 넣는다.

$$\mu'(s) = \arg \max_a \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_\mu(s') \}, \quad (8)$$

상태 전이가 결정적이면 안쪽 합이 사라져 다음이 된다.

$$\mu'(s) = \arg \max_a \{ r(s, a, f(s, a)) + \gamma v_\mu(f(s, a)) \}. \quad (9)$$

이렇게 만든 μ' 가 반드시 μ 이상이라는 것이 정책 개선 정리다. 앞 장에서 이 정리를 증명하고 3×4 그리드에서 한 번의 탐욕화가 10개 상태 전부의 가치를 올리는 것을 확인하였다. 동시에 그

한 번으로는 최적에 이르지 못하며 아래쪽 세 칸이 최적값에 최대 0.279 못 미친다는 것도 보았다. 이 절의 정책 반복법은 바로 그 간극을 메우는 절차다.

4.3.2 평가와 개선 반복

한 번 개선해 얻은 μ' 도 최적이 아니므로 다시 평가하고 다시 개선한다.

그림 4-14 정책 개선 과정

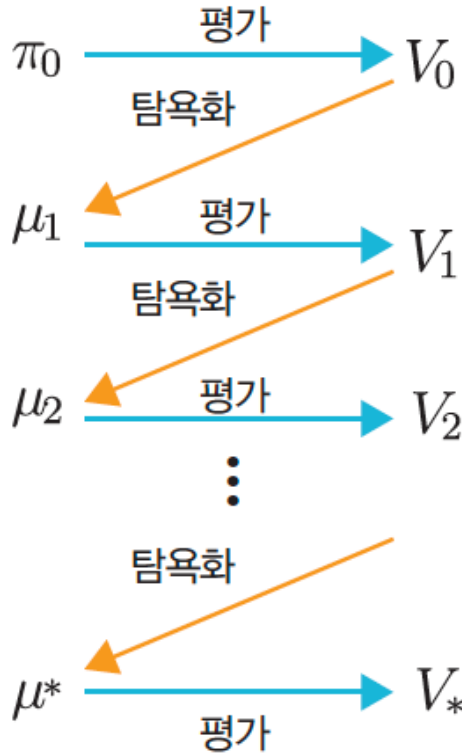


그림 12. 정책 반복법. 파란 화살표(평가)와 주황 화살표(탐욕화)가 번갈아 이어지며 내려간다. $\pi_0 \rightarrow V_0 \rightarrow \mu_1 \rightarrow V_1 \rightarrow \mu_2 \rightarrow \dots \rightarrow \mu_* \rightarrow V_*$. 출처: 원서 그림 4-14.

이 절차가 반드시 멈춘다는 것은 두 사실의 결합이다. 첫째, 정책 개선 정리에 의해 정책은 매번 나빠지지 않는다. 둘째, 결정적 정책의 수는 $|\mathcal{A}|^{|\mathcal{S}|}$ 로 유한하다. 개선될 때마다 서로 다른 정책으로 옮겨 가므로 유한 번 안에 더 이상 바뀌지 않는 정책에 도달한다. 그때는 $\mu = \text{greedy}(v_\mu)$ 이므로 벨만 최적 방정식이 성립하고, 따라서 $v_\mu = v_*$ 다.

여기서 유한성의 규모를 짚어 둘 필요가 있다. 3×4 그리드는 $|\mathcal{S}| = 12$, $|\mathcal{A}| = 4$ 이므로 $4^{12} = 16,777,216$ 가지 결정적 정책이 있다. 종료 보장이 말하는 상한은 이 숫자다. 그러나 실제로는 5회 만에 끝난다. 정책 반복법의 값어치는 종료가 보장된다는 사실이 아니라 그 보장을 훨씬 앞질러 끝난다는 데 있다.

4.4 정책 반복법 구현

앞 절의 절차를 코드로 옮긴다. 대상은 그림 13의 3×4 그리드 월드 그대로이고, 평가 부분은 §4.2.3의 `policy_eval()`을 고치지 않고 그대로 쓴다. 새로 필요한 것은 개선 단계 하나뿐이다.

그림 4-15 3×4 그리드 월드

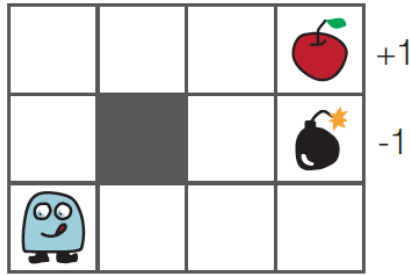


그림 13. 구현의 대상은 그대로 3×4 그리드 월드다. 평가할 정책만 무작위 정책에서 탐욕 정책으로 바뀐다. 출처: 원서 그림 4-15.

4.4.1 정책 개선

식 (9)을 그대로 옮긴 것이 `greedy_policy()`다. 각 상태에서 네 행동의 값을 모두 구한 뒤 가장 큰 것 하나에만 확률 1을 준다.

```
def greedy_policy(V, env, gamma):
    pi = {}
    for state in env.states():
        action_values = {}
        for action in env.actions():
            next_state = env.next_state(state, action)
            r = env.reward(state, action, next_state)
            action_values[action] = r + gamma * V[next_state]
        max_action = argmax(action_values)
        action_probs = {0: 0, 1: 0, 2: 0, 3: 0}
        action_probs[max_action] = 1.0 # 고른 행동에만 확률 1
        pi[state] = action_probs
    return pi
```

코드 6. 탐욕 정책 만들기 (s05_policy_iter.py)

결과는 결정적 정책이지만 형식은 그대로 {행동 : 확률}이다. 평가 코드를 고치지 않고 그대로 쓰기 위해서다. `argmax()`를 따로 만든 것은 값이 같은 행동이 여럿일 때 어느 것을 고를지 정해 두기 위함이며, 이 구현은 마지막에 나온 키를 고른다. 그래야 실행할 때마다 정책이 흔들리지 않는다. 이 결정은 §4.5 끝의 [보충]에서 최적 정책이 여러 개일 때 다시 문제가 된다.

4.4.2 평가와 개선 반복

그림 12을 그대로 옮기면 여덟 줄이 된다.

```
def policy_iter(env, gamma, threshold=0.001, is_render=True):
    pi = defaultdict(lambda: {0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25})
    V = defaultdict(lambda: 0)
    while True:
        V = policy_eval(pi, V, env, gamma, threshold) # 평가
        new_pi = greedy_policy(V, env, gamma) # 개선
        if new_pi == pi: # 정책이 바뀌지 않으면 종료
            break
        pi = new_pi
    return pi
```

코드 7. 정책 반복법 (s05_policy_iter.py)

코드 7에서 눈여겨볼 것은 v 를 사이클마다 새로 만들지 않고 이어 쓴다는 점이다. 이전 정책의 가치에서 출발하므로 두 번째 평가부터는 훨씬 빨리 수렴한다. 그 효과는 §4.5 끝의 [보충]에서 숫자로 확인한다.

결과

실행하면 평가와 개선의 사이클 5회 만에 정책이 고정된다.

표 6. 정책 반복법이 찾은 최적 가치와 최적 정책($\gamma = 0.9$, $\varepsilon = 10^{-3}$). (1, 1)은 벽이므로 계산은 되지 않지만 의미가 없다.

	최적 가치				최적 정책			
	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	0.81	0.90	1.00	0.00	RIGHT	RIGHT	RIGHT	GOAL
$h = 1$	0.73	(벽)	0.90	1.00	UP	WALL	UP	UP
$h = 2$	0.66	0.73	0.81	0.73	RIGHT	RIGHT	UP	LEFT

그림 4-16 처음과 마지막 가치 함수 및 정책(각 칸에 가치 함수의 값과 정책 표시)

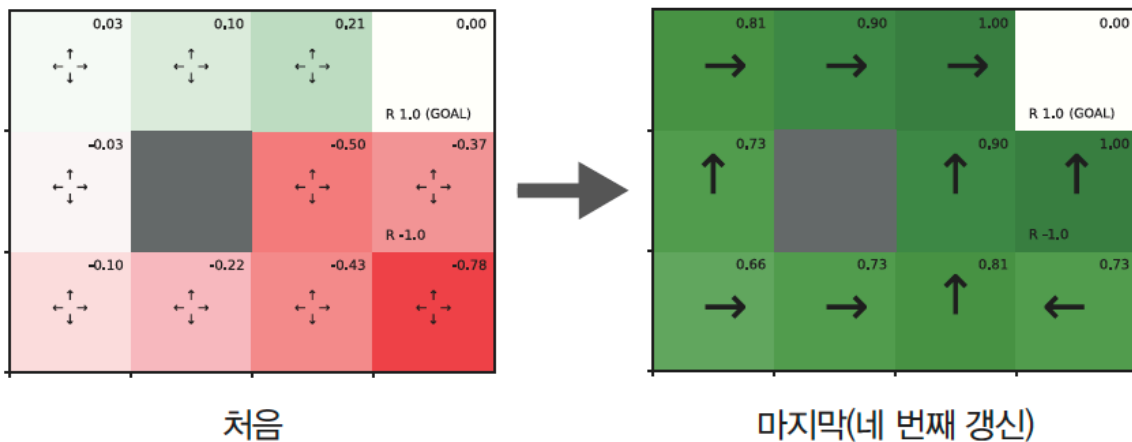


그림 14. 처음(왼쪽)과 마지막(오른쪽). 같은 환경인데 정책만 바뀌 색이 통째로 뒤집혔다. 무작위 정책의 (2, 3)은 -0.78 이었으나 최적 정책에서는 0.73 이다. 화살표도 사방에서 하나씩으로 줄었다. 출처: 원서 그림 4-16.

표 6의 값에는 규칙이 보인다. 목표 왼쪽 칸이 1.00 (한 걸음이면 $+1$), 그 왼쪽이 $0.90 = \gamma$, 그 왼쪽이 $0.81 = \gamma^2$ 이다. 목표까지의 걸음 수만큼 γ 가 곱해진 것이다. 오른쪽 아래 (2, 3)의 $0.73 = \gamma^3$ 은 폭탄을 피해 왼쪽으로 돌아가느라 네 걸음이 걸린다는 뜻이며, 폭탄 칸 바로 왼쪽 (1, 2)에서 화살표가 위를 향하는 것도 같은 이유다. 이 규칙을 일반화한 것이 §4.5 끝의 [보충] 최적 가치의 닫힌 형태다.

가치 함수가 고르게 짙은 초록인 이유는 최적 정책에서는 어느 칸에서 출발해도 사과에 닿기 때문이다. 무작위 정책에서는 폭탄에 빨려 들어가는 궤적이 섞여 값이 내려갔다.

4.5 가치 반복법

4.5.1 가치 반복법 도출

평가를 끝까지 돌릴 필요가 있는가

정책 반복법의 구조를 다시 보면 정책 μ 와 가치 V 사이를 평가와 개선이 오갈 뿐이다. 그림 15이 그 흐름과 한 바퀴의 속내다.

그림 4-20 반복적 정책 평가 알고리즘에 의한 갱신

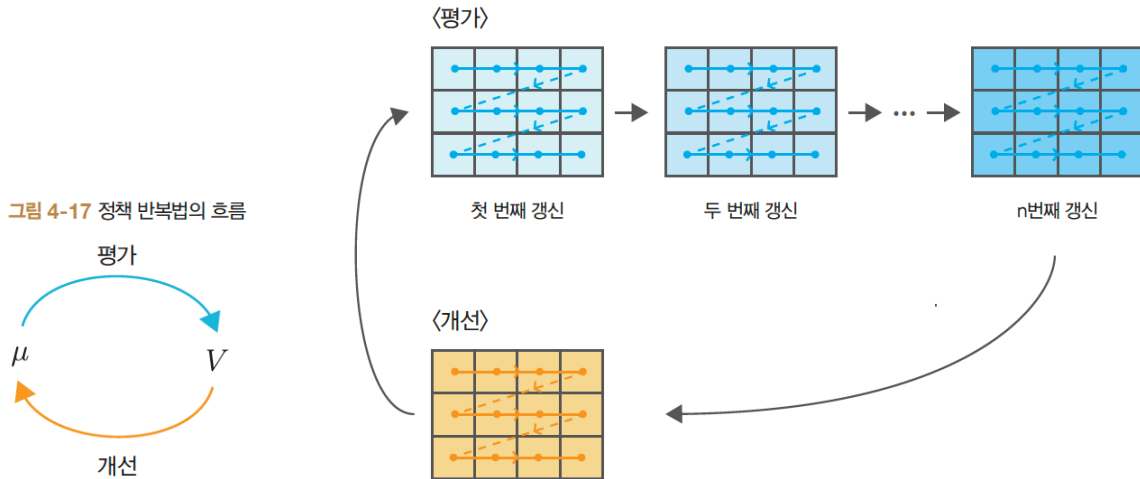


그림 15. 정책 반복법의 흐름(왼쪽)과 그 한 바퀴를 펼친 것(오른쪽). μ 와 V 사이를 평가(파랑)와 개선(주황)이 오간다. 오른쪽에서 보듯 지금까지의 정책 반복법은 파란 격자를 n 번 반복한 뒤에야 주황 격자로 넘어간다. 출처: 원서 그림 4-17, 4-20.

평가 단계는 문턱에 이를 때까지 여러 스윕을 돈다. 그런데 그 결과는 곧바로 탐욕화에 쓰일 뿐이고, 탐욕화가 필요로 하는 것은 정확한 값이 아니라 어느 행동이 가장 큰가라는 순서뿐이다. 그렇다면 평가를 끝까지 돌릴 필요가 있을까.

그림 4-18 정책 반복법에 의한 가치 함수와 정책의 개선 과정

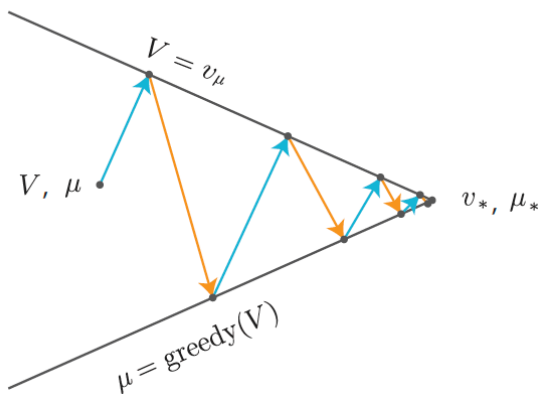


그림 4-19 가치 함수와 정책 개선 과정(변형)

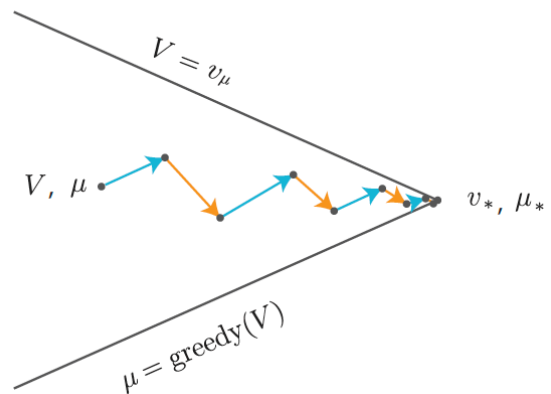


그림 16. 평가를 끝까지 돌리는 경우(왼쪽)와 도중에 끊는 경우(오른쪽). 왼쪽은 파란 화살표가 매번 위쪽 선 $V = v_\mu$ 에 정확히 닿는다. 오른쪽은 닿기 전에 꺾이지만 두 선이 만나는 꼭짓점 (v_*, μ_*) 으로 수렴하는 것은 마찬가지다. 화살표 하나하나를 짧아졌으나 개수가 늘었고, 전체 계산은 오히려 줄어든다. 출처: 원서 그림 4-18, 4-19.

극단으로 밀어 평가를 단 한 스윕만 하면 어떻게 되는가.

그림 4-21 각 상태에서 개선과 평가를 순차적으로 반복

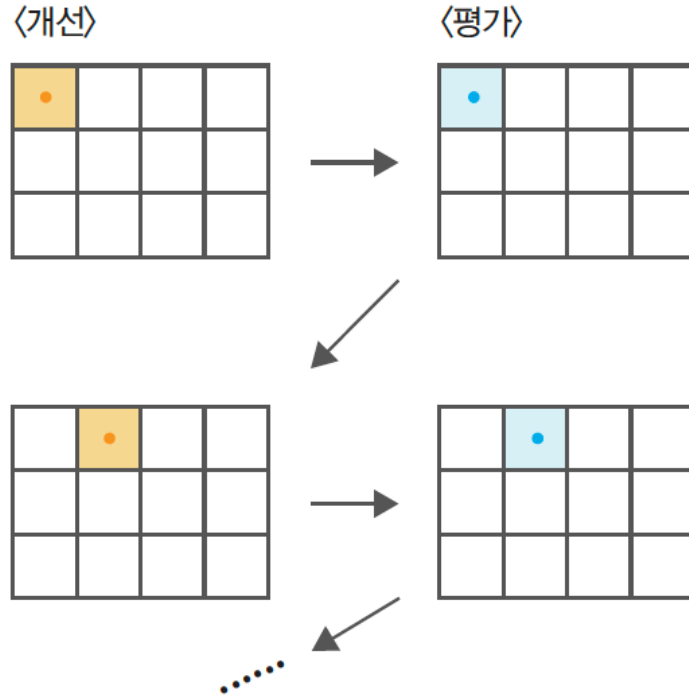


그림 17. 개선(주황)과 평가(파랑)가 한 칸 단위로 번갈아 일어나는 극단. 한 상태를 개선하고 곧바로 그 상태를 평가한 뒤 다음 상태로 넘어간다. 출처: 원서 그림 4-21.

두 단계는 같은 계산을 한다

이 극단에서 무슨 일이 일어나는지는 두 단계의 식을 나란히 놓으면 드러난다. 개선 단계에서 나온 정책은 결정적이므로 평가 쪽 식에서 $\sum_a \pi(a | s)$ 가 사라지고, 남는 것은 $\sum_{s'} p(s' | s, a) \{r + \gamma V(s')\}$ 하나다. 그런데 이는 개선 단계가 $\arg \max$ 를 취하는 대상과 완전히 같은 식이다.

$\arg \max$ 와 그 자리의 값을 한꺼번에 얻으려면 $\max$ 하나면 된다. 두 단계를 합친 갱신식은 다음과 같다.

$$V'(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma V(s')\} \xrightarrow{\text{결정적 전이}} \max_a \{r + \gamma V(f(s, a))\}. \quad (10)$$

식 (10)은 앞 장의 벨만 최적 방정식과 같은 모양이며 정책이 등장하지 않는다. 이 갱신을 반복하는 것이 가치 반복법이다. 정책 평가의 식 (3)과 비교하면 $\sum_a \pi(a | s)$ 자리가 $\max_a$ 로 바뀐 것뿐이다. 그림 19이 결정적 전이일 때의 형태까지 내려간 것이다.

4.5.2 가치 반복법 구현

정책을 따로 유지하지 않으므로 코드도 짧아진다. 한 스윕은 다음과 같다.

```
def value_iter_onestep(V, env, gamma):
    for state in env.states():
        if state == env.goal_state:
            V[state] = 0
            continue
        action_values = []
        for action in env.actions():
```

그림 4-22 개선과 평가 단계에서 수행되는 계산

$$\begin{aligned}
 \langle \text{개선} \rangle \quad \mu(s) &= \arg \max_a \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma V(s')\} \\
 \langle \text{평가} \rangle \quad a &= \mu(s) \text{ 일 때} \\
 V'(s) &= \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma V(s')\}
 \end{aligned}$$

같은 계산

그림 18. 개선 단계와 평가 단계가 실제로 하는 계산. 하늘색으로 칠해진 부분이 같은 식이다. 개선은 그 값들의 $\arg \max$ (어떤 행동인지)를, 평가는 고른 행동의 그 값(얼마인지)을 쓴다. 두 번 계산할 이유가 없다. 출처: 원서 그림 4-22.

그림 4-23 상태 전이가 결정적인 경우의 가치 반복법에 의한 갱신식

$$\begin{aligned}
 V'(s) &= \max_a \sum_{s'} p(s'|s, a) \{r(s, a, s') + \gamma V(s')\} & [\text{식 4.11}] \\
 &\downarrow \text{상태 전이가 결정적} \\
 s' &= f(s, a) \text{ 일 때} \\
 V'(s) &= \max_a \{r(s, a, s') + \gamma V(s')\} & [\text{식 4.13}]
 \end{aligned}$$

그림 19. 합친 갱신식(위)과 상태 전이가 결정적일 때의 형태(아래). 식 (10)의 왼쪽과 오른쪽에 각각 대응하며, 아래쪽이 코드 8에 그대로 나타난다. 출처: 원서 그림 4-23.

```

next_state = env.next_state(state, action)
r = env.reward(state, action, next_state)
action_values.append(r + gamma * V[next_state])
V[state] = max(action_values) # 정책 평가와 다른 곳은 여기뿐
return V

```

코드 8. 가치 반복법의 한 스윙 (s06_value_iter.py)

표 7. 정책 평가와 가치 반복법의 차이. 코드에서 다른 곳은 한 줄이다.

	정책 평가	가치 반복법
행동을 다루는 법	$\sum_a \pi(a s) (\dots)$	$\max_a (\dots)$
필요한 것	정책 π	없음
수렴하는 값	v_π	v_*

최적 가치를 얻은 뒤에는 마지막에 한 번만 탐욕화해 정책을 뽑는다. §4.4.1의 `greedy_policy()`를 그대로 재사용한다.

실행하면 4스윙 만에 값이 더 이상 바뀌지 않는다. 정책 반복법이 사이클마다 평가를 여러 스윙 돌린 것과 대조된다. 얻은 값은 표 6의 최적 가치와 완전히 같으며, 이 “4”라는 숫자가 문턱과 무관하게 결정된다는 것을 이 절 끝의 [보충]에서 보인다.

그림 4-24 가치 반복법에 의한 처음과 마지막(세 번째) 가치 함수

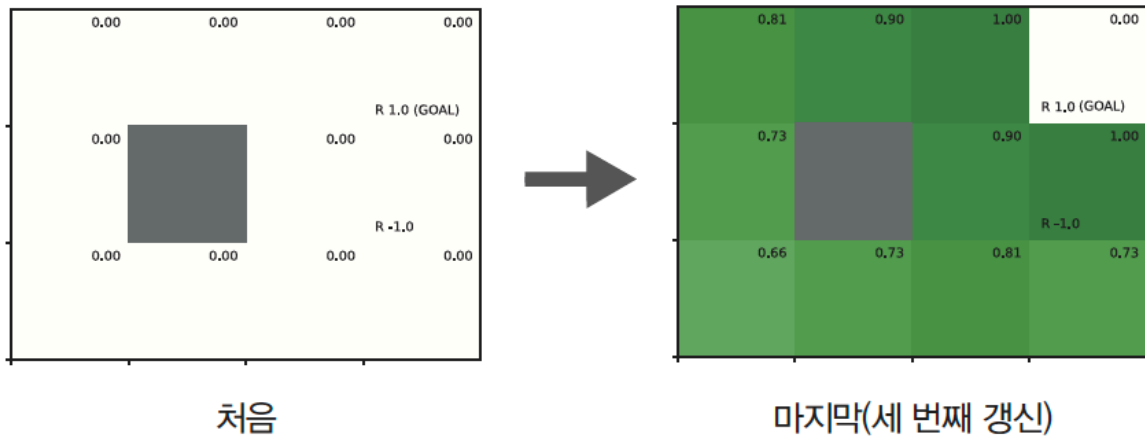


그림 20. 가치 반복법의 처음(왼쪽)과 마지막(오른쪽). 전부 0.00인 상태에서 출발해 세 번의 갱신으로 정책 반복법과 완전히 같은 값에 도달한다. 출처: 원서 그림 4-24.

[보충] 두 알고리즘의 실제 비용

원서의 비교표는 이 예제에서의 비용을 정책 반복법 “사이클 5회(내부 평가 포함)”, 가치 반복법 “스윙 4회”로 적는다. 괄호가 달려 있기는 하나 단위가 서로 달라 그대로 견줄 수 없다. 사이클 하나는 평가 스윙 여러 번을 품고 있기 때문이다. 같은 단위로 환산하면 표 8가 된다.

정책 반복법의 사이클별 평가 스윙 수는 23, 4, 2, 2, 1이다. 첫 사이클이 전체의 72%를 차지한다. 두 번째 사이클부터 급격히 줄어드는 데에는 두 가지 이유가 겹쳐 있는데, 어느 쪽이 주된 요인인지는 따로 재어 보아야 한다. 하나는 코드 7이 v 를 이어 쓰기 때문이고, 다른 하나는 두 번째 사이클부터는 평가 대상이 결정적 정책이기 때문이다. 사이클마다 v 를 0으로 되돌려 앞의 요인만 제거하면 스윙 수가 23, 4, 4, 4, 4로 바뀌어 총 39스윙이 된다. 즉 이어 쓰기가 절약하는 것은 7스윙(18%)에 그치며, 급감의 주된 요인은 결정적 정책의 평가가 $V = 0$ 에서 출발해도 네

그림 4-25 최적 상태 가치 함수로부터 얻은 최적 정책

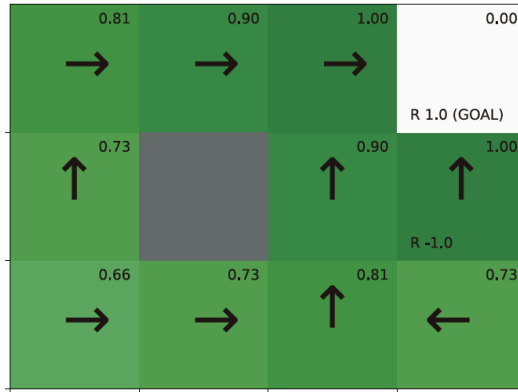


그림 21. 최적 가치 함수에서 뽑은 최적 정책. 그림 14 오른쪽과 화살표가 하나도 다르지 않다. 두 알고리즘의 답이 같다. 출처: 원서 그림 4-25.

표 8. 3×4 그리드에서 두 알고리즘의 비용 ($\gamma = 0.9$, $\varepsilon = 10^{-3}$). 한 스왑은 $|S||A| = 48$ 회의 백업이다.

	정책 반복법	가치 반복법	비
사이클(정책이 바뀐 횟수)	5	—	—
평가 스왑	32	4	8.0
탐욕화 횟수	5	1	5.0
백업 총계	1776	240	7.4

스왑이면 끝난다는 데 있다. 그 이유는 바로 다음 [보충]에서 밝힌다. 결정적 정책 아래의 가치도 γ 의 거듭제곱 꼴이어서 정보가 목표에서 몇 걸음만에 다 퍼지기 때문이다.

그렇다면 정책 반복법은 왜 쓰는가. 이 예제만 보면 답이 궁색하지만, 두 가지를 덧붙일 수 있다. 첫째, 정책 반복법은 사이클마다 쓸 수 있는 정책을 내놓는다. 계산을 중간에 멈춰도 그 시점의 μ_k 는 μ_{k-1} 보다 나쁘지 않은 실행 가능한 정책이다. 가치 반복법의 중간 산출물 V_k 는 어떤 정책의 가치도 아니다. 둘째, 상태 수가 크고 행동 수가 작을 때 평가 단계는 max가 없어 선형이므로 직접 해법으로 대체할 수 있다. $|S|$ 이 작으면 앞 장처럼 그 정책의 연립방정식을 한 번 푸는 편이 여러 스왑보다 빠르다.

결국 두 알고리즘은 “평가를 얼마나 끝까지 돌리는가”라는 하나의 축의 양 끝이다. 끝까지 돌리면 정책 반복법, 한 스왑만 돌리면 가치 반복법이며, 그 사이 어디에 두어도 최적으로 수렴한다. 이 둘을 일반화된 정책 반복(generalized policy iteration, GPI)이라 부르며 5장 이후의 알고리즘도 모두 이 안에 있다.

[보충] 최적 가치의 닫힌 형태와 유한 수렴

표 6에서 관찰한 “목표까지의 걸음 수만큼 γ 가 곱해진다”는 규칙은 이 환경에서 정확히 성립한다. $d(s)$ 를 “폭탄 칸 (1, 3)에 들어가지 않고 목표에 이르는 최단 걸음 수”라 하면, 벽과 목표를 제외한 모든 상태에서

$$v_*(s) = \gamma^{d(s)-1} \quad (11)$$

이다. 보상이 목표에 들어갈 때 +1, 폭탄에 들어갈 때 -1, 그 밖에는 0이므로 이유는 셋으로 갈린다. 폭탄을 피해 k 걸음에 목표에 닿는 경로의 수익은 γ^{k-1} 이고 그중 가장 큰 것이 $k = d(s)$ 일 때다. 폭탄을 한 번이라도 지나는 경로는 먼저 받는 -1이 나중에 받는 +1보다 덜 깎이므로 수익이 음수가 된다. 목표에 닿지 못하는 경로의 수익은 0 이하이다. 따라서 최댓값은 폭탄을 피하는 최단 경로가 주는 $\gamma^{d(s)-1}$ 이며, 12개 상태 전부에서 유리수 산술로 구한 정확해가 이 값과 일치함을 확인하였다.

표 9가 $d(s)$ 와 그로부터 나온 값이다.

표 9. 폭탄을 피하는 최단 걸음 수 $d(s)$ 와 최적 가치 $\gamma^{d(s)-1}$. 오른쪽 표가 표 6의 최적 가치와 소수점 아래까지 같다.

	$d(s)$				$\gamma^{d(s)-1}$			
	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	3	2	1	0	0.8100	0.9000	1.0000	0.0000
$h = 1$	4	—	2	1	0.7290	(백)	0.9000	1.0000
$h = 2$	5	4	3	4	0.6561	0.7290	0.8100	0.7290

표 9에서 두 칸을 따로 읽어야 한다. 폭탄 칸 (1, 3)의 $d = 1$ 은 그 칸에서 출발하는 경우이며, 보상이 들어갈 때 발생하므로 이미 폭탄에 서 있는 것은 벌점을 받지 않는다. 위로 한 걸음이면 목표이므로 $v_* = \gamma^0 = 1$ 이다. 한편 (2, 3)은 폭탄을 지나면 두 걸음이지만 $d = 4$ 로 적혀 있다. 폭탄을 지나는 값 $-1 + \gamma \cdot 1 = -0.1$ 이 돌아가는 값 $\gamma^3 = 0.729$ 보다 나쁘기 때문이다.

식 (11)에는 알고리즘에 대한 귀결이 하나 달려 있다. 최적 가치가 γ 의 유한한 거듭제곱이므로, 갱신이 그 값을 한 번 맞히면 더 이상 변하지 않는다. 즉 가치 반복법은 문턱과 무관하게 유한 스왑 안에 v_* 에 정확히 도달한다. 동기 갱신(배열 둘)은 $\max_s d(s) = 5$ 스왑, 제자리 갱신은 3스왑이면 값이 더 이상 바뀌지 않는다.

동기 갱신은 한 스왑마다 정보가 목표에서 한 걸음씩 퍼지므로 가장 먼 상태 (2, 0)의 $d = 5$ 가 그대로 스왑 수가 된다. 제자리 갱신은 같은 스왑 안에서 갱신된 값이 곧바로 쓰이므로 그림 10의 훑는 순서를 따라 여러 걸음이 한꺼번에 전파되어 3스왑으로 줄어든다. 예제 s06_value_iter.py가 4스왑에서 멈추는 것은 여기에 “값이 변하지 않았음”을 확인하는 스왑 한 번이 더해진 것이며, 원서 그림 4-24의 설명이 “마지막, 세 번째 갱신”이라고 적은 것과 정확히 맞는다. 이때 얻은 값과 정확한 v_* 의 차이는 1.11×10^{-16} 로 배경밀도의 분해능이다. 즉 이 문제에서 문턱 10^{-3} 은 실제로는 아무것도 자르지 않았다.

최적 정책이 여럿인 경우. 식 (11)은 최적 정책이 유일하지 않을 수 있음도 알려 준다. 어떤 상태에서 d 를 하나 줄이는 이웃이 둘이면 두 행동의 값이 같아지기 때문이다. 이 환경에서 그런 칸은 정확히 하나, (2, 0)이다. 위로 가면 (1, 0), 오른쪽으로 가면 (2, 1)인데 둘 다 $d = 4$ 이므로 양쪽 값이 $\gamma \cdot \gamma^3 = \gamma^4 = 0.6561$ 로 같다. 표 6이 그 칸에 RIGHT를 적은 것은 $\operatorname{argmax}()$ 가 동점일 때 마지막 키를 고르도록 만들어졌기 때문이며, 행동 번호가 UP(0), DOWN(1), LEFT(2), RIGHT(3)이므로 RIGHT가 남는다. 동점 처리 규칙을 바꾸면 그 칸의 화살표는 달라지지만 가치 함수는 달라지지 않는다.

4.6 정리

본고는 벨만 방정식을 반복 계산으로 푸는 세 알고리즘을 정리하였다.

출발점은 식 (1)의 등호를 대입으로 바꿔 읽는 것이었다. 우변에 추정치 V_k 를 넣어 좌변을 V_{k+1} 로 삼는 갱신(식 (2))은 참값과의 거리를 한 번에 γ 배 이하로 줄이므로 유일한 고정점으로 수렴한다. 이 갱신을 주어진 정책에 적용하면 반복적 정책 평가이고, 여기에 탐욕화를 번갈아 붙이면 정책 반복법이며, 평가를 한 스왑으로 줄여 개선과 합치면 $\sum_a \pi(a | s)$ 자리가 $\max_a$ 로 바뀐 가치 반복법(식 (10))이 된다. 세 알고리즘은 별개가 아니라 “평가를 얼마나 끝까지 돌리는가”라는 하나의 축 위의 서로 다른 지점이며, 그 축 전체가 일반화된 정책 반복이다.

본문 안에 “[보충]”으로 남긴 논의는 다섯이다.

첫째, “동적 프로그래밍”이라는 이름은 벨만이 예산을 얻기 위해 고른 것이라 기법의 성질과 관계가 없고, programming은 코딩이 아니라 당시 운용과학에서 쓰던 “계획 수립”이다. 내용 쪽의 정의는 최적 부분 구조와 중복되는 부분 문제이며, 앞의 것이 벨만 방정식, 뒤의 것이 여러 상태가 같은 s' 을 공유하는 사정, 그리고 표가 V 배열이다. 교과서 동적 계획법과 달리 되풀이가 필요한 것은 부분 문제의 의존 관계에 고리가 있어 먼저 확정할 순서가 없기 때문이다.

둘째, 배열 하나로 제자리 갱신하는 구현이 빠른 것($76 \rightarrow 60$)은 수치해석의 야코비 반복과 가우스-자이델 반복의 차이이다. 새 값이 같은 스왑 안에서 곧바로 쓰이므로 정보가 한 스왑에 더 멀리 퍼진다. 이 이점은 문턱을 낮출수록 커져 스왑 수의 비가 0.84에서 0.77로 내려가지만, 스왑 순서에 의존하고 병렬화를 막는다는 대가를 치른다.

셋째, 정지 문턱 ε 이 남기는 오차는 문턱 자체가 아니라 그 $\gamma/(1-\gamma)$ 배까지 허용된다(식 (7)). $\gamma = 0.9$ 에서 계수가 9이므로 문턱보다 한 자릿수 큰 오차를 각오해야 하며, 배열 둘 구현의 실측 8.325×10^{-4} 가 상한과 유효숫자 네 자리까지 같다. 앞 장의 정확해 표와 이 장의 반복 계산 표가 세 칸에서 소수 둘째 자리까지 다른 것도 바로 이 오차다.

넷째, 두 알고리즘의 비용을 같은 단위로 환산하면 평가 스왑 32회 대 4회, 백업 1776회 대 240회로 가치 반복법이 이 예제에서 7.4배 싸다. 정책 반복법의 스왑은 23, 4, 2, 2, 1로 첫 사이클에 몰려 있다. 그럼에도 정책 반복법이 갖는 이점은 중간에 멈춰도 실행 가능한 정책이 손에 남는다는 것과, 평가 단계가 선형이라 직접 해법으로 대체할 수 있다는 것이다.

다섯째, 이 환경의 최적 가치는 $v_*(s) = \gamma^{d(s)-1}$ 이라는 닫힌 형태를 가지며(d 는 폭탄 칸에 들어가지 않는 최단 걸음 수), 그로부터 가치 반복법이 유한 스왑 안에 정확히 끝남이 따라온다. 동기 갱신은 $\max_s d(s) = 5$ 스왑, 제자리 갱신은 3스왑이며, 예제가 4스왑에서 멈추는 것은 확인 스왑 한 번이 더해진 결과다. 이 문제에서 문턱 10^{-3} 은 실제로는 아무것도 자르지 않았고, 얻은 값은 배경밀도 한계인 1.11×10^{-16} 까지 정확하다.

이 장의 모든 방법은 환경 모델 p 와 r 를 완전히 알고 있어야 한다. 코드가 `next_state()`와 `reward()`를 직접 호출한다는 사실이 그 전제의 표현이다. 현실 문제에서는 모델을 모르는 경우가 대부분이고 상태 수도 감당하기 어렵다. 다음 장에서는 모델 없이 직접 경험한 표본으로 가치를 추정하는 몬테카를로법을 다룬다.

재현 정보

본고의 모든 수치는 Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1 환경에서 재현하였다. 예제는 난수를 쓰지 않으므로 시드 고정 없이 항상 같은 값이 나온다.

표 10. 예제 코드 목록. 앞의 여섯은 원서 4장의 예제이고 마지막은 본고의 그림 생성 스크립트다.

파일	내용	대응 절
ch04/s01_dp.py	배열 둘을 쓰는 반복적 정책 평가	§4.1.2
ch04/s02_dp_inplace.py	제자리 갱신	§4.1.3
ch04/s03_gridworld_play.py	<code>render_v()</code> 시각화 확인	§4.2.1
ch04/s04_policy_eval.py	3×4 그리드의 반복적 정책 평가	§4.2.3
ch04/s05_policy_iter.py	정책 반복법	§4.4
ch04/s06_value_iter.py	가치 반복법	§4.5.2
ch04/paper/fig_convergence.py	그림 7 생성	§4.1 끝의 [보충]

표 1, 6과 §4.2.3의 스왑 수는 표 10의 예제를 그대로 실행해 얻은 출력이다. 표 2, 3, 4, 8, 9은 본고를 위해 별도로 계산하였다.

표 2는 코드 1과 2의 정지 문턱만 바꿔 실행한 것이다. 표 8의 스왑 수는 `policy_eval()` 호출마다 내부 반복 횟수를 세어 합한 값이며, 백업 총계는 스왑 수에 $|S||\mathcal{A}| = 48$ 을 곱한 것이다. 표 9의 $d(s)$ 는 폭탄 칸으로 들어가는 이동을 막은 너비 우선 탐색으로 구하였고, 식 (11)과 유한 수렴은 `fractions.Fraction` 산술로 값이 완전히 고정될 때까지 반복하여 확인하였다. 부동소수점을 쓰지 않았으므로 “더 이상 변하지 않는다”가 반올림의 결과가 아니다.

3×4 그리드의 상태는 원서와 예제 코드의 표기를 따라 (h, w) 로 적었다. h 가 위에서부터 센 행, w 가 왼쪽에서부터 센 열이며 (0, 3)이 목표, (1, 1)이 벽, (1, 3)이 폭탄이다. 표에서 벽 칸의 값을 비워 둔 것은 `env.states()`가 벽도 내놓아 계산은 되지만 갈 수 없는 칸이라 의미가 없기 때문이다. 참고로 그 값은 0.81로, 벽을 보통 칸으로 취급했을 때의 γ^2 과 같다.

참고문헌

- [1] 사이토 고키, 밑바닥부터 시작하는 딥러닝 4: 파이썬으로 구현하는 강화 학습 알고리즘, 한빛 미디어, 2023.
- [2] sajacaros, 벨만 방정식과 최적성: 무한한 미래를 유한한 연립방정식으로 바꾸는 재귀 구조, 2026.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [4] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- [5] R. Bellman, *Eye of the Hurricane: An Autobiography*, World Scientific, 1984.
- [6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.
- [7] R. S. Varga, *Matrix Iterative Analysis*, 2nd ed., Springer, 2000.

본 문서에 실린 그림 중 “원서 그림”으로 표기한 것은 참고문헌 [1]의 도판이며, 학습 정리 목적으로 인용하였다. 그 밖의 그림과 표는 본고에서 직접 생성한 것이다.